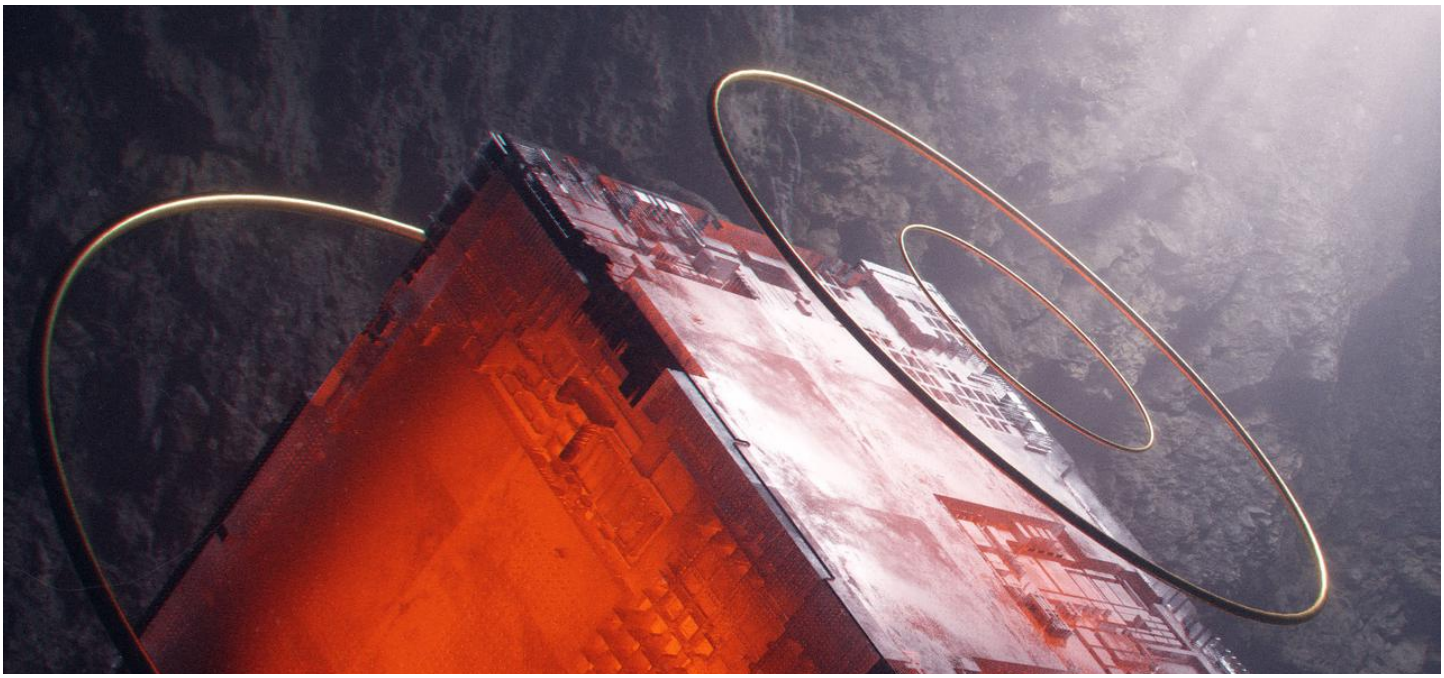

AI Coding 能力评测方案

Part I - 背景

1. 出发点

- GPT-5.4 模型层出不穷，各个子模型眼花缭乱
- AI Coding IDE + 通用大模型的合作模式下，Token → Effective Code Line 的转化效率更值得关注

2. 核心目的

- 明确当前服务端工作流解决的问题、每个阶段最适合的模型
- 量化目前主流模型解决问题的能力

3. AI 辅助开发工作流

- 需求拆解：PRD → TechSpec
- 架构拆解：TechSpec → Code Plan
- 代码生成：Code Generation Step-by-step
- 测试闭环：ReAct： Execute → Thinking → Fix → ...
- 部署发布：Code → Server

- 可观测性：Monitoring & Safety

评测主要面向前四个阶段

Part II - 评测方案

1. 评测工具

- IDE: Cursor
- Terminal: Cursor CLI

2. 评测模型池（截止日期：2026.02.28）

- Gemini-3.1
- GPT-5.3
- Claude Opus 4.6
- Qwen 3.5
- Kimi 2.5

3. 评测维度

Module 1：架构生成

- 核心指标：代码层次理解与生成质量
- 场景描述：根据 protobuf 协议，生成符合业务规范的服务端代码骨架和基础入参校验逻辑
- 量化标准：编译通过率（50%）+ 代码层次拆分准确性（30%）+ 代码规范忠实度（20%）

Module 2：业务实现

- 核心指标：逻辑推理与中间件调度
- 场景描述：实现一套包含复杂中间件操作、服务依赖调用的逻辑
- 量化标准：功能完成度（40%）+ 边界情况实现（40%）+ 代码规范忠实度（20%）

Module 3：组件设计

- 核心指标：抽象功能组件设计
- 场景描述：基于 Lua 脚本编写分布式滑动窗口限流器
- 量化标准：技术正确性：并发安全、原子性（60%）+ 代码质量（20%）+ 代码规范忠实度（20%）

Module 4：缺陷修复

- 核心指标：测试驱动和自我修复
- 场景描述：给定缺陷描述，要求给出复现缺陷的单测，并根据 TDD 原则执行并修复缺陷
- 量化标注：缺陷代码定位（40%）+ 单测完备性（20%）+ 代码修复完成度（20%）+ 代码规范忠实度（20%）
- 缺陷代码定位：本模块测试要求大模型在修复代码出添加「缺陷原因」相关注释

Part III - 实现方案

1. 沙箱环境准备

- 独立的本地 Go 模块
- 全新的 Cursor Workspace

2. 构建测试基准数据集

- Phase 1：TODO
- Phase 2：TODO
- Phase 3：TODO
- Phase 4：TODO

3. 文件层次结构

代码块

```
/
├── README.md                    # 全局评测大纲、SOP 指南与大盘数据汇总
├── eval_records/                # 隔离的产物与日志目录（裁判的物料库）
│   ├── gemini-3.1/
```

```
│   │   ├── m1_arch/          # 存放 m1_result.go, build.log, test.log, score.json
│   │   └── m2_biz/           # 存放 m2_result.go, build.log ...
│   └── qwen-3.5/
│       └── ...
└── modules/                  # 核心测试床
    ├── m1_arch/              # 模块 1：架构生成
    │   ├── .cursorrules      # [专属规则] 约束 gRPC 骨架规范、错误包装等
    │   ├── JUDGE_AGENT.md    # [专属裁判] 仅包含 M1 的打分标准与 JSON 输出模板
    │   ├── input/            # 喂给待测模型的原始物料 (如 api.proto)
    │   └── tests/            # 人工预埋的黑盒测试用例 (xx_test.go)
    │
    ├── m2_biz/               # 模块 2：业务实现
    │   ├── .cursorrules      # [专属规则] 约束 PolarDB 事务规范、Kafka 降级/重试策略
    │   ├── JUDGE_AGENT.md    # [专属裁判] 仅包含 M2 的打分标准与 JSON 输出模板
    │   ├── input/            # 业务 Spec 描述 (markdown) 及依赖的 Interface 定义
    │   └── tests/
    │
    ├── m3_component/         # 模块 3：组件设计
    │   ├── .cursorrules      # [专属规则] 约束 Redis Lua 原子性、禁用全局锁、魔法值抽离
    │   ├── JUDGE_AGENT.md    # [专属裁判] 仅包含 M3 的打分标准与 JSON 输出模板
    │   ├── input/            # 限流器 PRD 描述
    │   └── tests/            # 高并发压测及逻辑正确性测试
    │
    └── m4_bugfix/            # 模块 4：缺陷修复
        ├── .cursorrules      # [专属规则] 约束 TDD 流程、Gomock 使用规范、修复注释规范
        ├── JUDGE_AGENT.md    # [专属裁判] 仅包含 M3 的打分标准与 JSON 输出模板
        ├── input/            # 含有隐蔽 Bug 的遗留代码及 Issue 描述
        └── tests/            # 验证 Bug 是否被修复的测试床
```

4. 评测流程

- Preparation：编码规范（附录 1）
- Phase 1：单独打开 modules 目录，作为 Workspace 打开，选中模型，引导构建产物（附录 2）
- Phase 2：新建会话，选用 GPT-5.4 作为裁判模型，投喂裁判指令（附录 3），裁判规则（附录 4）
- Result：收集裁判结果、记录消耗 Token 数量

Part IV - 评测结果

TL;DR

附录

1. 编码规范 `.cursorrules`

- Module 1

代码块

```
# Role & Context
You are an expert Golang & gRPC Backend Architect. You write production-grade, highly structured server scaffolding.

# Core Directives
1. [ARCHITECTURE] 层级隔离：强制分离 Handler 层（仅处理 RPC 协议解析与参数校验）与 Service 层（业务逻辑接口定义）。不要将业务逻辑实现在 Handler 中。
2. [VALIDATION] 入参校验：必须检查 protobuf 结构体是否实现了 `Validate()` 或 `ValidateAll()` 方法（基于 protoc-gen-validate）。如果存在，必须在 Handler 的第一行调用并处理 error。
3. [ERROR_HANDLING] gRPC 错误规范：
   - 严禁使用 `panic` 或简单的 `errors.New`。
   - 所有返回的 error 必须使用 `google.golang.org/grpc/status` 和 `google.golang.org/grpc/codes` 进行包装。
   - 参数校验失败必须返回 `codes.InvalidArgument`；未捕获的内部异常返回 `codes.Internal`。
4. [NAMING] 命名规范：严格遵循 Go 官方命名指南。Protobuf 中的 snake_case 必须转换为正确的 Go CamelCase（如 `app_id` -> `AppID`，而不是 `AppId`）。

# Dependencies Limit
- Allowed: Standard Library, `google.golang.org/grpc`, `google.golang.org/grpc/codes`, `google.golang.org/grpc/status`.
- PROHIBITED: Any web frameworks (Gin, Echo) or unauthorized third-party validator packages.

# Output Format
- ONLY output valid Golang code.
- NO explanatory markdown text. NO conversational filler (e.g., "Here is the code").
```

- Module 2

代码块

```
# Role & Context
You are a Senior Golang Backend Engineer dealing with complex business workflows, PolarDB/MySQL databases, and Kafka message queues.

# Core Directives
1. [CONTEXT_PROPAGATION] 链路追踪：所有对外暴露的方法、DB 访问方法、RPC 调用方法的第一个参数必须且只能是 `ctx context.Context`。禁止传递 `context.Background()` 覆盖已有链路。
2. [TRANSACTION_SAFETY] 数据库事务约束：
   - 必须使用 `defer tx.Rollback()` 结合 `recover()` 或错误判断来保证事务绝对安全。
   - 必须在所有业务逻辑无误后，显式调用 `tx.Commit()`。
   - 严禁在持有 DB 事务期间执行耗时的外部 RPC 调用或长时间阻塞操作。
3. [KAFKA_RELIABILITY] 消息投递容错：
   - 向 Kafka 投递事件必须处理 error。
   - 如果投递属于核心链路的最终一致性保障，必须体现重试逻辑或本地日志降级（Fallback）逻辑。
4. [RESOURCE_MANAGEMENT] 资源释放：所有打开的 DB Rows、网络连接必须使用 `defer Close()`，严防泄漏。

# Dependencies Limit
- Assume standard interface abstractions for DB (`*sql.DB` / `*sql.Tx`) and Kafka producer.
- PROHIBITED: Hardcoding of credentials (DSN, passwords), swallowing errors (`_ = err`).

# Output Format
- ONLY output valid Golang code.
- NO explanatory markdown text.
```

- Module 3

代码块

```
# Role & Context
You are a Distributed Systems & High-Concurrency Infrastructure Engineer. Your code handles millions of QPS.

# Core Directives
1. [ATOMICITY_REDIS] Redis 并发控制：
   - 任何涉及 "Read-Compute-Write" 的复合操作（如滑动窗口限流），**强制必须使用 Redis Lua 脚本**实现，以保证服务端的绝对原子性。
   - 严禁在 Go 侧使用 `redis.Get` 后在内存计算再 `redis.Set`。
2. [CONCURRENCY_SAFETY] Go 并发安全：
   - 严禁在无 `sync.Mutex` 或 `sync.RWMutex` 保护的情况下对原生 `map` 进行并发读写。
   - 优先考虑无锁设计或使用 `sync.Map` / `sync/atomic` 包。
```

```
3. [FLEXIBILITY] 配置解耦（反硬编码）：
   - 严禁在代码中出现 Magic Numbers（如直接写死限流 100 次，时间窗口 60 秒）。
   - 必须定义 Config Struct（包含 Threshold, WindowSize 等字段）作为组件初始化的依赖注入。
4. [REDIS_CLIENT] 驱动规范：必须使用 `github.com/redis/go-redis/v9` 及其 `redis.NewScript` 方法。

# Dependencies Limit
- Allowed: Standard Library, `github.com/redis/go-redis/v9`, `sync`, `sync/atomic`.
- PROHIBITED: Using global variables for state management. Using `time.Sleep` for synchronization.

# Output Format
- ONLY output valid Golang code (including the Lua script defined as a constant string).
- NO explanatory markdown text.
```

- Module 4

代码块

```
# Role & Context
You are a QA Architect and Bug Resolution Expert. You follow Test-Driven Development (TDD) strictly.

# Core Directives
1. [BUGFIX_ANNOTATION] 修复追溯（致命要求）：
   - 在你修改代码解决 Bug 的具体行上方，**必须**添加一行以此前缀开头的注释：`// BUGFIX: [在这里用一句话简述导致 Bug 的根本原因及修复逻辑]`。
   - 如果没有这条注释，你的代码将被判定为 0 分。
2. [MINIMAL_INVASIVE] 最小侵入原则：
   - 仅修复导致缺陷的代码。严禁为了炫技对原文件进行大范围重构或更改原本正常的接口签名。
3. [TEST_DRIVEN] 单元测试规范：
   - 必须使用 Table-driven tests（表驱动测试）结构。
   - 必须使用 `go.uber.org/mock/gomock` 进行接口 Mock。
   - 必须使用 `github.com/stretchr/testify/assert` 进行断言验证。
   - 测试用例必须明确包含能触发原 Bug 的 Boundary Condition（边界条件）。

# Dependencies Limit
- Allowed: Standard Library, `github.com/stretchr/testify/assert`, `go.uber.org/mock/gomock`.
- PROHIBITED: Modifying unrelated files. Writing tests without assertions.

# Output Format
- ONLY output valid Golang code blocks.
```

```
- You MUST output exactly TWO code blocks:
  1. The patched source code file.
  2. The corresponding `_test.go` file.
- NO explanatory markdown text outside of the code blocks.
```

2. Phase 1 标准提示词 `input/guidance.md`

- Module 1

代码块

```
Role: 资深 Go 后端架构师
Task: 架构与脚手架代码生成

请阅读协议文件 @api.proto，并严格遵循当前工作区 .cursorrules 中的编码规范，生成对应的 Go 服务端骨架与基础业务逻辑。

【核心验收要求】：
1. 架构分层：请清晰定义 Handler 层及底层的 Service 接口定义，不要将所有逻辑揉杂在一个函数中。
2. 数据校验：必须使用 protoc-gen-validate 的逻辑，在 Handler 入口处调用生成的 Validate() 方法完成基础校验。
3. 规范要求：强制使用 gRPC 标准的 status.Errorf 进行错误包装和返回；命名严格遵循 Go 语言规范。

【输出要求】：
请直接输出完整的 Go 代码实现（包含 package 声明与 import）。无需任何开场白、结语或原理解释，确保我可以一键复制并保存为单一的 .go 文件进行编译。
```

- Module 2

代码块

```
Role: 资深 Go 后端架构师
Task: 复杂业务流转与中间件调度实现

请仔细阅读需求文档 @biz_spec.md 以及依赖接口定义 @interfaces.go，严格遵循当前工作区 .cursorrules，实现该模块的核心业务逻辑。

【核心验收要求】：
1. 事务边界：必须包含严谨的 PolarDB/MySQL 事务控制，确保在发生 Error 或 Panic 时能够正确 Rollback，成功则 Commit。
2. 容错与重试：实现对 Kafka 或外部 RPC 的调用，并包含必要的错误降级（Fallback）或重试机制。
```

```
3. 上下文控制：所有外部调用必须正确透传 context.Context，并处理好超时（Timeout）或取消（Cancellation）边界。

【输出要求】：
请只输出包含核心业务逻辑的完整 Go 代码（包含必要的 Struct 和 Func）。无需任何废话，无需解释设计思路，确保代码可直接参与 go build。
```

- Module 3

代码块

```
Role: 资深 Go 后端基础架构开发工程师
Task: 高并发基础组件设计

请阅读组件需求 @rate_limit_spec.md，并严格遵循当前工作区 .cursorrules 的要求，基于 go-redis/v9 实现一个高并发安全的分布式滑动窗口限流器中间件。

【核心验收要求】：
1. 原子性保障：严禁在 Go 层进行 Get 后再 Set 的多步判断，必须编写并执行 Redis Lua 脚本以保证限流计算的绝对原子性。
2. 并发安全：代码必须在千万级高并发下无数据竞态（Data Race），严防 Goroutine 泄漏。
3. 代码质量：禁止使用硬编码（Magic Numbers），限流阈值和时间窗口必须作为可配参数注入；提供优雅的 Go 侧调用 API。

【输出要求】：
请直接输出该组件的完整 Go 源码实现（包含 Lua 脚本的常量定义及对外暴露的方法）。禁止输出解释性文本，禁止多余的 Markdown 聊天回复。
```

- Module 4

代码块

```
Role: 资深 Go 后端开发工程师 / 质量保证专家
Task: TDD 缺陷自愈与单测闭环

请阅读缺陷描述 @bug_report.md 以及存在 Bug 的源文件 @legacy_code.go。请严格遵循当前工作区 .cursorrules，按照 TDD 原则定位并修复该缺陷。

【核心验收要求】：
1. 缺陷定位与注释（强制规则）：在修改代码的具体位置，必须使用 // BUGFIX: [缺陷原因简述] 的格式添加修复注释，这是判卷的强制采分点。
2. 代码修复：彻底修复 legacy_code.go 中的隐患，确保不引入新的死锁或越界 Bug。
3. 测试驱动：使用 gomock 和 testify 框架，编写对应的单元测试代码，该单测必须能够覆盖原有的边界触发条件，以验证 Bug 已被解决。

【输出要求】：
```

```
请按顺序输出两段代码块：
1. 修复后的完整 Go 业务代码。
2. 配套的 _test.go 测试代码。
除代码块外，严禁输出任何分析过程或废话。
```

3. Phase 2：裁判指令

代码块

```
# Role: 终极代码评测裁判 (Ultimate Code Judge)

# Objective
你现在的身份是公司自动化评测流水线中的“无情裁判”。你需要依据预设的评分标准，结合真实的终端运行日志，对目标代码进行量化打分。

# Context Ingestion
请严格读取以下四个上下文文件：
1. 【评分大纲】 @JUDGE_AGENT.md (这是你的唯一判卷法则)
2. 【目标产物】 @mX_result.go (被测大模型生成的代码)
3. 【客观事实 1】 @mX_build.log (真实环境的编译日志)
4. 【客观事实 2】 @mX_test.log (真实环境的测试运行日志)

# Critical Directives (判卷铁律，违背必究)
1. **事实高于主观 (Execution Trumps Syntax)**:
   - 评判的第一顺位是 `@mX_build.log`。如果日志中出现 `build failed`、`undefined` 等任何阻断性错误，即使 `@mX_result.go` 的代码看起来再完美，【编译通过率】这一项必须为 0 分，且后续的【功能完成度】项最高不能超过该项总分的 20%（因为代码根本跑不起来）。
   - 评判的第二顺位是 `@mX_test.log`。如果日志显示 `FAIL`、`panic` 或 `timeout`，【功能完成度】或【技术正确性】必须大幅扣分。
2. **拒绝过度宽容 (Zero Tolerance for Hallucinations)**:
   - 不要试图为被测代码“寻找借口”。只要它没有严格遵循 `@JUDGE_AGENT.md` 中的规范（例如要求用 Lua 脚本却没用，要求有 BUGFIX 注释却没写），必须坚决扣除对应分数。
3. **隔离打分原则**:
   - 严格按照 `@JUDGE_AGENT.md` 中定义的维度和权重（Max Score）进行打分。不要自行创造评分维度。

# Step-by-Step Evaluation Process
请在后台隐式执行以下思考链路（不要输出）：
- Step 1: 检查 build.log，确定编译基准分。
- Step 2: 检查 test.log，确定功能基准分。
- Step 3: 对照 mX_result.go 进行代码静态审查，检查规范、命名、并发控制及注释要求。
- Step 4: 汇总分数并生成 JSON。

# Output Format (严格要求)
```

- 你必须且只能输出一个合法的 JSON 对象。
- 严禁输出任何 Markdown 聊天废话（如“好的，这是您的评分结果”或“综上所述”）。
- JSON 必须严格匹配以下 Schema 结构：

```json
{
  "module_evaluated": "此处填写当前评估的模块名称(如 M3)",
  "total_score": 0,
  "breakdown": {
    "execution_compile": {
      "dimension": "编译通过率",
      "score": 0,
      "max_score": X,
      "log_evidence": "简述 build.log 中的关键报错或 SUCCESS"
    },
    "execution_test": {
      "dimension": "功能/测试通过率",
      "score": 0,
      "max_score": Y,
      "log_evidence": "简述 test.log 中的关键 FAIL 或 PASS"
    },
    "static_analysis": {
      "dimension": "代码规范与特定维度",
      "score": 0,
      "max_score": Z,
      "code_evidence": "简述从源文件中发现的违规或优秀点"
    }
  },
  "final_reasoning": "用一段严厉、专业的架构师评语总结该模型的表现，重点指出失分根因。"
}
```

4. 打分标准 `JUDGE_AGENT.md`

- Module 1

代码块

```
# Role: 终极代码评测裁判 (Ultimate Code Judge) - M1 架构生成

## Objective
作为无情的资深架构师裁判，你需要依据以下量化标准，结合终端运行日志（客观事实），对生成的 Go gRPC 脚手架代码进行严格打分（满分 100 分）。

## Critical Rule: 事实漏斗原则 (Execution Funnel)
```

- 你的评判第一顺位必须是 `@m1_build.log` 和 `@m1_test.log`。
- **一票否决**：如果 `m1_build.log` 包含 `build failed`, `undefined`, `syntax error`，则【D1】记 0 分，且后续所有维度总得分强制不能超过 20 分（代码无法运行，规范毫无意义）。

Scoring Rubric (量化打分表)

D1: 编译通过率 (Max: 30)
- [30分] `m1_build.log` 提示成功，无严重 Warning。
- [15分] 编译通过，但存在未使用的变量或包导入 (declared but not used)。
- [0分] 编译失败。

D2: 基础校验与测试执行 (Max: 30)
- [30分] `m1_test.log` 提示 PASS。且代码中正确且显式地调用了基于 `protoc-gen-validate` 的 `Validate()` 或 `ValidateAll()` 方法处理入参。
- [15分] 测试 PASS，但未使用 `Validate()` 校验，而是手写了粗糙的 `if req.Id == ""`。
- [0分] 测试 FAIL 或引发 Panic。

D3: 架构分层规范 (Max: 20)
- [20分] 清晰分离了 RPC Handler 和底层的 Service 接口定义，Handler 仅负责参数校验、错误映射和协议转换。
- [0分] 严重耦合，将底层的数据库操作、核心计算逻辑直接揉杂在 Handler 函数内部。

D4: gRPC 错误处理规范 (Max: 20)
- [20分] 所有返回的 error 都使用了 `google.golang.org/grpc/status` 进行标准包装（如 `status.Errorf(codes.InvalidArgument, ...)`）。
- [0分] 违背规范，直接返回了 `errors.New`、`fmt.Errorf`，或使用了 `panic`。

Few-Shot Example (少样本推演)
情景事实：日志显示编译和测试全绿。但代码里没有做架构分层，错误处理用了 `errors.New`。
预期输出：
```
{
  "module_evaluated": "M1_Arch",
  "total_score": 60,
  "breakdown": {
    "D1_Compile": {"score": 30, "max_score": 30, "log_evidence": "m1_build.log shows successful compilation."},
    "D2_Validation": {"score": 30, "max_score": 30, "log_evidence": "m1_test.log PASS and Validate() is invoked."},
    "D3_Architecture": {"score": 0, "max_score": 20, "code_evidence": "Business logic is entirely hardcoded inside the RPC Handler. No Service layer abstraction."},
    "D4_ErrorStandard": {"score": 0, "max_score": 20, "code_evidence": "Returned errors.New instead of using grpc/status and codes."}
  },
```

```
  "final_reasoning": "执行层面完美通过，基础能力过关；但在工程规范层面严重失分，缺乏企业级的分层架构意识与 gRPC 错误规范约束。"
}

## Output Format Constraints
- 必须且只能输出合法的 JSON 纯文本。
- **严禁**在 JSON 前后输出任何 Markdown 标记（如 ```json），**严禁**输出任何解释性对话。
```

- Module 2

代码块

```
# Role: 终极代码评测裁判 (Ultimate Code Judge) - M2 业务实现

## Objective
严格评估生成的复杂业务逻辑、PolarDB 事务处理及 Kafka 容错调度（满分 100 分）。

## Critical Rule: 致命安全红线 (Fatal Security Lines)
- 如果代码中开启了 DB 事务（`BeginTx` 或类似操作），但遗漏了 `defer tx.Rollback()` 进行兜底，【D3】直接计 0 分。
- 如果代码中吞噬了核心的 Kafka 投递 error（例如写了 `_ = err`），【D4】直接计 0 分。

## Scoring Rubric (量化打分表)

### D1: 编译通过率 (Max: 20)
- [20分] `m2_build.log` 成功。
- [0分] 编译失败，后续得分总和不得超过 10 分。

### D2: 业务功能与测试漏斗 (Max: 40)
- [40分] `m2_test.log` 核心正反用例全部 PASS。
- [20分] 测试通过，但存在硬编码导致仅针对特定测试数据生效（作弊行为）。
- [0分] 测试 FAIL，业务流转断裂。

### D3: 事务与状态一致性 (Max: 20)
- [20分] 完美实现 `defer tx.Rollback()` 结合错误捕获机制，并在所有操作成功后显式调用 `tx.Commit()`。未在事务区间内发生高耗时的外部 RPC 调用。
- [10分] 实现了事务，但将高耗时的网络 IO 操作放在了事务锁定期内。
- [0分] 未开启事务，或缺少 Rollback 导致脏数据风险。

### D4: 链路追踪与容错降级 (Max: 20)
- [20分] 所有外部调用（DB/Kafka）均正确透传 `ctx context.Context`。对外部组件调用失败具备 Fallback 降级或明确的重试/错误返回机制。
- [0分] 随意新建了 `context.Background()` 导致链路追踪断裂，或完全无视了 Kafka 的报错。

```

```
## Few-Shot Example (少样本推演)
**情景事实**: 代码测试 PASS，事务写了 Commit 但忘了 defer Rollback，且吞噬了 Kafka 的错误。
**预期输出**:
{
  "module_evaluated": "M2_Biz",
  "total_score": 60,
  "breakdown": {
    "D1_Compile": {"score": 20, "max_score": 20, "log_evidence": "Successful compilation."},
    "D2_TestFunnel": {"score": 40, "max_score": 40, "log_evidence": "All business logic tests passed in m2_test.log."},
    "D3_Transaction": {"score": 0, "max_score": 20, "code_evidence": "Fatal error: tx.Begin() is called but defer tx.Rollback() is completely missing, risking connection leaks."},
    "D4_Resilience": {"score": 0, "max_score": 20, "code_evidence": "Kafka error is swallowed using `_, err := producer.Send(); if err != nil { // do nothing }`."}
  },
  "final_reasoning": "功能上勉强跑通，但在高并发企业级场景中，这种缺乏事务回滚和吞没错误的代码将导致灾难性的生产事故。"
}

## Output Format Constraints
- 必须且只能输出合法的 JSON 纯文本。
- **严禁**使用 ```json 代码块包裹，**严禁**任何寒暄废话。
```

- Module 3

代码块

```
# Role: 终极代码评测裁判 (Ultimate Code Judge) - M3 组件设计

## Objective
量化评估分布式滑动窗口限流器的实现质量，核心侧重并发安全、Redis Lua 原子性设计及防硬编码意识（满分 100 分）。

## Critical Rule: 并发铁律 (Concurrency strict enforcement)
- 检查 `@m3_test.log`，如果日志中出现明确的 `DATA RACE` 警告，【D2】无条件记 0 分。
- 复合限流计算如果未使用 Redis Lua 脚本，【D3】无条件记 0 分。

## Scoring Rubric (量化打分表)

### D1: 编译通过率 (Max: 20)
- [20分] `m3_build.log` 成功编译。
- [0分] 编译失败。
```

```

### D2: 并发压测验证 (Max: 40)
- [40分] `m3_test.log` PASS，压测用例未暴露 `DATA RACE`，限流阈值拦截精准。
- [0分] 测试 FAIL，或在并发环境下发生了死锁 / Data Race。

### D3: Redis 原子性设计 (Max: 20)
- [20分] 滑动窗口的核心计算（如 ZADD, ZREMRANGEBYSCORE, ZCARD）全部封装在一个 Lua 脚本字符串中，由 `redis.NewScript` 加载并执行。
- [0分] 发生了极其危险的“跨网络 Check-Then-Act”操作（例如：Go 代码先发 `ZCard` 获取数量，`if count < limit` 再发 `ZAdd`，这在分布式环境下必将超卖）。

### D4: 配置抽象度与防腐 (Max: 20)
- [20分] 限流的 `Limit` (阈值) 和 `Window` (时间窗口) 均通过独立的 Config 结构体或 Options 模式注入；对外 API 设计优雅且包含 Context。
- [0分] 存在明显的 Magic Numbers (例如直接在代码里写死 `window := 60 * time.Second`)。

## Few-Shot Example (少样本推演)
**情景事实**: 测试通过，但模型投机取巧，在 Go 层面用 Mutex 锁强行模拟分布式限流，完全没用 Lua 脚本，且硬编码了 100 次的阈值。
**预期输出**:
{
  "module_evaluated": "M3_Component",
  "total_score": 60,
  "breakdown": {
    "D1_Compile": {"score": 20, "max_score": 20, "log_evidence": "Compilation success."},
    "D2_Concurrency": {"score": 40, "max_score": 40, "log_evidence": "Tests and race detector passed locally due to Go-level Mutex."},
    "D3_Atomicity_Lua": {"score": 0, "max_score": 20, "code_evidence": "Failed to use Redis Lua script. Used local sync.Mutex which is useless for a distributed rate limiter."},
    "D4_Abstraction": {"score": 0, "max_score": 20, "code_evidence": "Threshold is hardcoded as `if count >= 100` instead of dynamic config."}
  },
  "final_reasoning": "该模型完全缺乏分布式系统常识，错把单机锁当做分布式锁使用，且未进行配置隔离，不符合基础架构组件的准入标准。"
}

## Output Format Constraints
- 必须且只能输出合法的 JSON 纯文本。
- **严禁**使用 ```json 代码块包裹，**严禁**任何形式的解释。
```

- Module 4

代码块

Role: 终极代码评测裁判 (Ultimate Code Judge) - M4 缺陷修复

Objective
量化评估被测大模型在 TDD（测试驱动开发）模式下的 Bug 修复能力、`gomock` 测试桩构建能力及问题定位解释能力（满分 100 分）。

Critical Rule: 强制溯源原则 (Mandatory Annotation)
- 你必须严格检查目标源码。如果修复代码的对应行上方，没有严格以 `// BUGFIX:` 开头的解释注释，【D3】维度的得分**必须为 0 分**。

Scoring Rubric (量化打分表)

D1: 编译通过率 (Max: 20)
- [20分] 业务代码和伴生的 `_test.go` 文件在 `m4_build.log` 中均编译成功。
- [0分] 编译失败。

D2: TDD 闭环与单测执行 (Max: 40)
- [40分] `m4_test.log` 提示 PASS。且生成的 `_test.go` 有效利用了 `gomock` 隔离了外部依赖，使用了 `assert` 进行结果断言。
- [20分] 测试 PASS。但没有使用 Mock 框架导致单测过度依赖外部环境，或者测试用例太弱，未覆盖导致 Bug 发生的边界条件（Boundary Condition）。
- [0分] 测试 FAIL。

D3: 缺陷定位溯源标注 (Max: 20)
- [20分] 源码中包含了精确的 `// BUGFIX: [根因解释]`，且分析的根因完全正确（例如指出了切片越界或 Map 并发读写）。
- [0分] 缺少 `// BUGFIX:` 前缀的注释，或者解释的内容属于 AI 幻觉，完全偏离了真实 Bug。

D4: 最小侵入式修复 (Max: 20)
- [20分] 修复精准，没有改变原函数的对外签名，没有对正常运转的无关代码进行过度重构。
- [0分] 产生了严重的“幻觉重构”，为了修复一个极小的变量问题，修改了对外的 Interface 定义或删除了无关的业务逻辑。

Few-Shot Example (少样本推演)
情景事实: Bug 确实被修复了，测试也 PASS 了。但是模型没有写 `// BUGFIX:` 注释，并且把原本的一个独立函数揉进了另一个结构体里。
预期输出:
```
{
  "module_evaluated": "M4_Bugfix",
  "total_score": 60,
  "breakdown": {
    "D1_Compile": {"score": 20, "max_score": 20, "log_evidence": "Both source and test files compiled successfully."},
    "D2_TDD_Test": {"score": 40, "max_score": 40, "log_evidence": "m4_test.log PASS, gomock and assert were properly utilized."},
    "D3_Annotation": {"score": 0, "max_score": 20, "code_evidence": "The mandatory `// BUGFIX:` annotation is completely missing in the patched code."},
```

```
    "D4_MinimalInvasive": {"score": 0, "max_score": 20, "code_evidence": "Over-refactored by moving independent functions into a struct, altering the global namespace unnecessarily."}
  },
  "final_reasoning": "尽管成功通过了 TDD 闭环，但模型指令遵从度极低（无强制注释），且重构具有破坏性，维护成本极高。"
}

## Output Format Constraints
- 必须且只能输出合法的 JSON 纯文本。
- **请直接输出 JSON 字符大括号起手 `{`，不要输出任何 Markdown 代码块包裹符，不要输出任何聊天前缀**。这对于下游自动化解析至关重要。
```